Modules in Node.js — Complete Deep Dive

Start From The Very Beginning — What Problem Do Modules Solve?

Imagine you are building a large application. Let's say you are building an e-commerce website. This application needs to handle:

- User authentication (login, signup, logout)
- Product listings
- Shopping cart
- Payment processing
- Order management
- Email notifications
- Database connections

Now imagine writing ALL of this code in a **single file**. One file. Everything together.

That file would have thousands and thousands of lines. Finding a specific function would be like finding a specific word in a novel without an index. Fixing a bug in the payment section might accidentally break something in the cart section because everything is tangled together. If two developers try to work on the same file at the same time, they constantly conflict with each other.

This is not hypothetical — this is exactly what happened in early programming. And it was a nightmare.

Modules are the solution.

A module is simply this: **a separate file that contains a specific piece of your application's functionality.**

Instead of one massive file, you have:

```
auth.js        → handles login, signup, logout
products.js    → handles product listings
cart.js        → handles shopping cart
payments.js    → handles payment processing
orders.js      → handles order management
email.js       → handles email notifications
database.js    → handles database connections
```

Each file is focused on one thing. Each file is a module.

But now a new question arises: if your code is split across many files, **how does one file use code from another file?**

That is the entire purpose of the module system — to answer this question.

What a Module System Does

A module system provides two fundamental operations:

1. Export — marking certain pieces of code in a file as available for other files to use

2. Import — bringing code from another file into the current file so you can use it

Without these two operations, each file is completely isolated. With them, your files can communicate and share code with each other in a controlled, organised way.

The word "controlled" is important here. Not everything in a file needs to be available to other files. Maybe you have helper functions that are only used internally within that file. The module system lets you choose exactly what to expose and what to keep private.

The Three Types of Modules in Node.js

Before going into CommonJS vs ES Modules, you need to understand that there are three categories of modules in Node.js:

1. Built-in Modules (Core Modules)

These come pre-installed with Node.js. You don't need to download or install anything. They are built into Node.js itself.

Examples:

- `fs` — for working with the file system
- `os` — for operating system information
- `http` — for creating web servers
- `path` — for working with file paths
- `crypto` — for cryptography

You can use them immediately after installing Node.js.

2. User-Defined Modules

These are files YOU create in your project. The `auth.js`, `products.js`, `cart.js` examples above — those are user-defined modules. You write them, you decide what they export, you import them wherever you need them.

3. Third-Party Modules

These are packages created by other developers and published to the npm registry. When you run `npm install express`, you are downloading a third-party module. You then import it into your files just like you would a user-defined module.

Now The Big Question — What Are CommonJS and ES Modules?

Here is the core thing to understand:

CommonJS and ES Modules are two different SYNTAXES for doing the same thing — exporting and importing code.

They are two different ways to write the same operations. Think of them like two different dialects of the same language. Both accomplish the goal of sharing code between files, but they use different keywords and have different behaviours under the hood.

To understand why two systems exist, you need to understand the history.

The History — Why Two Systems Exist

JavaScript Was Originally Browser-Only

When JavaScript was first created in 1995, it only ran in browsers. And in browsers, there was no concept of files importing other files. All your JavaScript was loaded via `<script>` tags in HTML. Everything shared a single global scope. There were no modules.

This was fine for small scripts but became a problem as web applications grew more complex.

CommonJS Was Created for Node.js

When Ryan Dahl created Node.js in 2009, he needed a module system. JavaScript had none built in. So the Node.js team adopted a specification called **CommonJS** — a community-created standard for how JavaScript modules should work on the server.

CommonJS gave Node.js a module system using `require()` and `module.exports`. This worked well and became deeply embedded in the entire Node.js ecosystem. Every package on npm used it. Millions of lines of code were written with it.

JavaScript Finally Got Its Own Module System

In 2015, the ECMAScript specification (the official JavaScript standard) introduced **ES Modules** (ESM) — a native module system built into JavaScript itself. Not just Node.js, not just browsers — JavaScript as a language now had modules.

ES Modules use `import` and `export` keywords.

This created a problem: Node.js already had CommonJS deeply embedded. The entire npm ecosystem was built on it. You couldn't just switch overnight. So Node.js had to support BOTH systems simultaneously.

This is why two systems exist today. Not because someone wanted complexity — but because of historical evolution.

CommonJS — The Old System

The Core Idea

In CommonJS, every file is automatically wrapped in a function by Node.js behind the scenes. This function receives some special variables, one of which is `module`. The `module` object has a property called `exports`. Whatever you assign to `module.exports` becomes what other files receive when they `require()` your file.

Exporting in CommonJS

Exporting a single thing:

```
// greet.js
function greet(name) {
   return "Hello " + name
}

module.exports = greet
```

Here you are saying: when someone requires this file, give them the `greet` function directly.

Exporting multiple things:

```
// mathOperations.js
function add(a, b) {
   return a + b
}

function subtract(a, b) {
   return a - b
}

function multiply(a, b) {
   return a * b
}

module.exports = {
   add: add,
   subtract: subtract,
   multiply: multiply
}
```

Here you are exporting an object that contains multiple functions. The person importing this file gets an object and can access each function through the object's properties.

Shorthand — exports:

There is also a shorthand called just `exports` (without `module.`):

```
// mathOperations.js
exports.add = function(a, b) {
   return a + b
}

exports.subtract = function(a, b) {
   return a - b
}
```

This works the same way for adding properties. However there is one critical rule: **never reassign `exports` directly.** Only add properties to it.

```
// This BREAKS everything - never do this
exports = function greet() {}  // Wrong

// This is fine
exports.greet = function() {}  // Correct
```

The reason is technical — `exports` is just a reference to `module.exports`. If you reassign `exports` itself, you break that reference and nothing gets exported.

Importing in CommonJS

```
// app.js
const greet = require('./greet')

console.log(greet("Rakesh"))  // Hello Rakesh
```

The `require()` function takes a path and returns whatever that file exported via `module.exports`.

```
// app.js
const math = require('./mathOperations')

console.log(math.add(5, 3))       // 8
console.log(math.subtract(10, 4)) // 6
console.log(math.multiply(3, 3))  // 9
```

Importing built-in modules:

```
const fs = require('fs')
const os = require('os')
const path = require('path')
```

No `./` prefix for built-in modules — Node.js knows to look in its core library.

Importing third-party modules:

```
const express = require('express')
const axios = require('axios')
```

Same syntax — no `./` prefix. Node.js looks in `node_modules` folder.

Destructuring on import:

```
const { add, subtract } = require('./mathOperations')

console.log(add(5, 3))      // 8
console.log(subtract(10, 4)) // 6
```

How require() Works Internally

When you call `require('./greet')`, Node.js does the following steps:

1. **Resolves the path** — figures out the exact file location
2. **Checks the cache** — has this file been required before? If yes, return the cached result immediately (this is important — CommonJS caches modules)
3. **Reads the file** — reads the JavaScript file from disk
4. **Wraps it** — wraps your code in a function like this:

```
(function(exports, require, module, __filename, __dirname) {
   // YOUR CODE GOES HERE
})
```

5. **Executes it** — runs the wrapped function
6. **Returns module.exports** — gives back whatever was assigned to `module.exports`
7. **Caches it** — stores the result so future `require()` calls for the same file return immediately

This wrapping is why `module`, `exports`, `require`, `__filename`, and `__dirname` are available in every Node.js file without you importing them — they are injected by this wrapper function.

CommonJS is Synchronous

This is a critical technical detail. `require()` is **synchronous** — it blocks execution until the file is loaded and executed. When you call `require()`, your program stops and waits for that module to fully load before moving to the next line.

```
const fs = require('fs')         // stops here until fs is loaded
const express = require('express') // stops here until express is loaded

// only reaches here after both are loaded
const app = express()
```

This works fine at the top of a file during startup. But it would be a problem if you tried to `require()` something in the middle of handling a request — it would block your entire server.

ES Modules — The Modern System

The Core Idea

ES Modules (ESM) are the official, standardised module system built into JavaScript itself as part of the ECMAScript specification. They use `import` and `export` keywords that are part of the language syntax — not functions or objects like `require` and `module.exports`.

Exporting in ES Modules

There are two types of exports in ES Modules: **named exports** and **default exports**.

Named Exports

```
// mathOperations.js
export function add(a, b) {
   return a + b
}

export function subtract(a, b) {
   return a - b
}

export function multiply(a, b) {
   return a * b
}
```

You put the `export` keyword directly in front of whatever you want to export. You can have as many named exports as you want in a single file.

You can also export at the bottom of the file:

```
// mathOperations.js
function add(a, b) {
   return a + b
}

function subtract(a, b) {
   return a - b
```

```
}

function multiply(a, b) {
   return a * b
}

export { add, subtract, multiply }
```

Both approaches do exactly the same thing.

Default Export

```
// greet.js
function greet(name) {
   return "Hello " + name
}

export default greet
```

Or written more concisely:

```
// greet.js
export default function greet(name) {
   return "Hello " + name
}
```

The critical rule about default exports:

Each file can have **only one** default export. Only one. This makes sense because the default export represents the primary thing that file is about. If a file needs to export one main thing, use default export.

Named exports vs Default export — when to use which:

- Use **default export** when your file has one primary thing to export
- Use **named exports** when your file exports multiple things of equal importance

Importing in ES Modules

Importing named exports:

```
// app.js
import { add, subtract, multiply } from './mathOperations.js'
```

```
console.log(add(5, 3))       // 8
console.log(subtract(10, 4)) // 6
```

The curly braces `{}` are required for named imports. The names inside the braces must **exactly match** the names used when exporting. This is what "named" means — the name is part of the contract.

Importing a default export:

```
// app.js
import greet from './greet.js'

console.log(greet("Rakesh"))  // Hello Rakesh
```

No curly braces for default imports. And you can name it **anything you want**:

```
import myGreetingFunction from './greet.js'
import sayHello from './greet.js'
import g from './greet.js'
```

All three of these import the same default export. The name you give it is just your local variable name.

Importing both default and named from the same file:

```
import greet, { add, subtract } from './utils.js'
```

The default import comes first, then named imports in curly braces.

Importing everything from a file:

```
import * as math from './mathOperations.js'

console.log(math.add(5, 3))
console.log(math.subtract(10, 4))
```

The `* as math` syntax imports all named exports and puts them into an object called `math`.

Importing built-in modules:

```
import { readFile, writeFile } from 'node:fs/promises'
```

```
import * as os from 'node:os'
import path from 'node:path'
```

Note the `node:` prefix — this is the modern recommended way to import built-in modules. It makes it immediately clear this is a Node.js built-in, not a third-party package.

Importing third-party modules:

```
import express from 'express'
import axios from 'axios'
import { useState, useEffect } from 'react'
```

No path prefix needed — Node.js looks in `node_modules`.

Important — file extensions in ES Modules:

Unlike CommonJS where you could omit `.js`:

```
const math = require('./mathOperations') // works in CommonJS
```

In ES Modules you must include the file extension:

```
import { add } from './mathOperations.js' // must include .js
```

ES Modules are Asynchronous

This is the opposite of CommonJS. ES Modules load **asynchronously**. Node.js can start loading multiple modules simultaneously without waiting for each one to finish before starting the next.

This makes ES Modules more aligned with how Node.js works generally — the non-blocking, asynchronous approach.

It also means you can use `import` inside async functions in certain ways, and there is a feature called **dynamic import** which we'll look at next.

Dynamic Import — Import Inside Functions

Standard `import` statements must be at the top of the file — they are static. But sometimes you need to load a module conditionally or only when needed. For this there is dynamic import:

```
async function loadModule() {
```

```
  if (someCondition) {
    const { add } = await import('./mathOperations.js')
    console.log(add(5, 3))
  }
}
```

Dynamic import uses `import()` as a function (with parentheses) and returns a Promise. This is useful for performance — only loading modules when actually needed rather than loading everything at startup.

The package.json `"type"` Field — The Missing Piece

Now we get to the exact thing you asked about in the question. This is where most people get confused.

The Problem

Node.js needs to know which module system to use when it reads your `.js` files. A `.js` file could be CommonJS or ES Modules — the syntax looks different, but Node.js needs to decide before it starts parsing.

What `"type": "module"` Does

In your `package.json`:

```
{
  "name": "my-project",
  "version": "1.0.0",
  "type": "module"
}
```

This single line tells Node.js: **treat every `.js` file in this project as an ES Module.**

This means:

- You can use `import` and `export` syntax in your `.js` files
- `require()` will NOT work in `.js` files
- `module.exports` will NOT work in `.js` files
- Node.js expects ES Module syntax throughout

What Happens Without `"type": "module"`

If you do NOT include `"type": "module"` in your `package.json`, Node.js defaults to CommonJS mode.

```
{
  "name": "my-project",
  "version": "1.0.0"
}
```

This means:

- Every `.js` file is treated as CommonJS
- `require()` and `module.exports` work
- `import` and `export` keywords will cause a syntax error

Or you can explicitly set it:

```
{
  "name": "my-project",
  "version": "1.0.0",
  "type": "commonjs"
}
```

This does the same as having no `"type"` field — explicitly sets CommonJS as the default.

Overriding Per File

You can override the project-level setting for individual files using different extensions:

- `.mjs` — always treated as ES Module regardless of `package.json`
- `.cjs` — always treated as CommonJS regardless of `package.json`

```
project/
  package.json  ("type": "module")
  index.js      → ES Module (follows package.json)
  legacy.cjs    → CommonJS (overridden by extension)
  modern.mjs    → ES Module (always ES Module)
```

This is useful when mixing old and new code in the same project.

Why Does This Setting Exist at All?

Because millions of packages on npm are written in CommonJS. When Node.js added ES Module support, it had to maintain backwards compatibility. It couldn't just break every existing package.

The `"type"` field lets each project declare which system it uses, so Node.js can parse files correctly.

Side by Side Comparison

Here is the exact same code written in both systems so you can see the difference clearly:

File 1 — The Module Being Exported

CommonJS:

```
// userService.js
const users = []

function createUser(name, email) {
   const user = { id: Date.now(), name, email }
   users.push(user)
   return user
}

function getAllUsers() {
   return users
}

function getUserById(id) {
   return users.find(user => user.id === id)
}

module.exports = {
   createUser,
   getAllUsers,
   getUserById
}
```

ES Modules:

```
// userService.js
const users = []

export function createUser(name, email) {
    const user = { id: Date.now(), name, email }
    users.push(user)
    return user
}

export function getAllUsers() {
    return users
}

export function getUserById(id) {
    return users.find(user => user.id === id)
}
```

File 2 — The File That Uses It

CommonJS:

```
// app.js
const { createUser, getAllUsers, getUserById } = require('./userService')

createUser("Rakesh", "rakesh@example.com")
createUser("Priya", "priya@example.com")

console.log(getAllUsers())
```

ES Modules:

```
// app.js
import { createUser, getAllUsers, getUserById } from './userService.js'

createUser("Rakesh", "rakesh@example.com")
createUser("Priya", "priya@example.com")

console.log(getAllUsers())
```

Key Differences Summarised

Aspect	CommonJS	ES Modules
Syntax — export	`module.exports = ...`	`export` / `export default`
Syntax — import	`require()`	`import ... from`
Loading	Synchronous (blocking)	Asynchronous (non-blocking)
When resolved	At runtime	At parse time (before execution)
Dynamic imports	Always dynamic	Static by default, dynamic via `import()`
File extension	`.js` or `.cjs`	`.js` (with `"type":"module"`) or `.mjs`
`package.json`	No `"type"` or `"type":"commonjs"`	`"type":"module"`
`__dirname`	Available automatically	Not available (must use `import.meta.url`)
`__filename`	Available automatically	Not available (must use `import.meta.url`)
Age	2009 (Node.js creation)	2015 (ES6 specification)
Status	Legacy but widely used	Modern standard

Which One Should You Use?

Use ES Modules for new projects

ES Modules are the official JavaScript standard. They are the future. All modern tooling, frameworks, and the JavaScript specification itself is moving towards ES Modules. If you are starting a new project today, use ES Modules.

Set this in your `package.json`:

```
{
  "type": "module"
}
```

And use `import`/`export` everywhere.

You will encounter CommonJS — know it well

The entire existing npm ecosystem — millions of packages — was written in CommonJS. Older codebases at companies are in CommonJS. When you join a company and look at existing code, you will very likely see `require()` and `module.exports`. You must understand it.

The simple mental model

CommonJS → require() and module.exports → old way → still works
ES Modules → import and export → new way → recommended

Both work. Both are valid. ES Modules is where everything is heading. CommonJS is what everything already is. Know both, default to ES Modules for new work.

export default vs export — Complete Deep Dive

First, Forget Everything and Start From Zero

Let me build this up so slowly and carefully that by the end it will feel completely obvious.

Imagine you have a file. That file has some code in it. Maybe functions, maybe variables, maybe objects. Now you want other files to be able to use that code.

The question is: **how do you make code available to other files?**

The answer is: **you export it.**

But here is where the confusion starts. There are **two completely different ways** to export things in ES Modules:

1. `export` — called **Named Export**
2. `export default` — called **Default Export**

They look similar. They both use the word `export`. But they behave completely differently and solve different problems.

Named Export — `export`

The Core Idea

A named export is exactly what it sounds like — you export something **with a specific name attached to it.**

That name becomes part of the contract. Whoever imports it must use that exact name.

How to Write a Named Export

```
// mathOperations.js

export function add(a, b) {
   return a + b
}

export function subtract(a, b) {
   return a - b
}

export function multiply(a, b) {
   return a * b
}
```

You put the keyword `export` directly in front of each thing you want to export. Each one gets exported with its own name — `add`, `subtract`, `multiply`.

These three names are now the contract. They are locked in.

How to Import Named Exports

```
// app.js
import { add, subtract, multiply } from './mathOperations.js'
```

See those **curly braces** `{}`? Those are required for named imports. And the names inside the curly braces must **exactly match** the names used during export.

If you exported `add`, you must import `add`. You cannot call it something else.

```
// This will FAIL
import { addition } from './mathOperations.js'
// Error: The requested module does not provide an export named 'addition'
```

The name `addition` does not exist as an export. The export was named `add`. So you must write `add`.

You Can Have UNLIMITED Named Exports

This is the key feature of named exports. **One file can have as many named exports as it wants.**

```
// utils.js

export const PI = 3.14159

export function greet(name) {
   return "Hello " + name
}

export function calculateArea(radius) {
   return PI * radius * radius
}

export const appName = "My Application"

export class User {
   constructor(name) {
      this.name = name
   }
}
```

Five exports from one file. All named. All available to anyone who imports from this file.

When importing, you pick only what you need:

```
// I only need greet and PI
import { greet, PI } from './utils.js'

// I only need calculateArea
import { calculateArea } from './utils.js'

// I need everything
import { PI, greet, calculateArea, appName, User } from './utils.js'
```

Default Export — `export default`

The Core Idea

A default export is completely different in philosophy. It says: **this file has ONE main thing. This is the primary thing this file is about.**

There is no name attached to the export itself. The file is just saying "here is my main thing, take it."

How to Write a Default Export

```
// greet.js

function greet(name) {
   return "Hello " + name
}

export default greet
```

Or written in one line:

```
// greet.js

export default function greet(name) {
   return "Hello " + name
}
```

Notice what is different:

- No curly braces
- The word `default` after `export`
- Only ONE thing is being exported as default

The Absolute Rule About Default Export

Each file can have MAXIMUM ONE default export. Only one. Never more than one.

This will cause an error:

```
// WRONG - you cannot have two default exports
export default function greet() {}
export default function farewell() {} // SyntaxError
```

Node.js will throw a syntax error immediately. Two default exports in one file is simply not allowed.

How to Import a Default Export

```
// app.js
import greet from './greet.js'
```

Notice what is different here compared to named imports:

1. **No curly braces** `{}` — named imports use curly braces, default imports do not
2. **You can name it ANYTHING you want**

```
import greet from './greet.js'
import sayHello from './greet.js'
import myFunction from './greet.js'
import xyz from './greet.js'
```

All four of these import the exact same thing — the default export from `greet.js`. The name you give it is completely up to you. It is just your local variable name for that thing.

This is fundamentally different from named exports where the name is fixed.

The Exact Example From The Transcript

The transcript used a specific example. Let me recreate it exactly and explain every single line.

The File Being Exported — greeter.js

```
// greeter.js

function getGreeting(hours) {
    if (hours < 4 || hours >= 19) {
        return "Good Night"
    } else if (hours < 9) {
        return "Good Morning"
    } else if (hours < 16) {
        return "Good Afternoon"
    } else {
        return "Good Evening"
```

```
    }
}

export default getGreeting
```

This file has one main function — `getGreeting`. The whole purpose of this file is this one function. So it makes sense to use `export default`.

Importing the Default Export

```
// app.js
import getGreeting from './greeter.js'

const hours = new Date().getHours()
const greeting = getGreeting(hours)
console.log(greeting)
```

No curly braces. The name `getGreeting` is chosen by us — we could have called it anything.

Now Using Named Export Instead

If we used named export on the same function:

```
// greeter.js

export function getGreeting(hours) {
    if (hours < 4 || hours >= 19) {
        return "Good Night"
    } else if (hours < 9) {
        return "Good Morning"
    } else if (hours < 16) {
        return "Good Afternoon"
    } else {
        return "Good Evening"
    }
}
```

Now importing it:

```
// app.js
import { getGreeting } from './greeter.js'
```

Curly braces required. Name must be exactly `getGreeting`.

The Transcript's Specific Error Example

The transcript showed something very important. Let me recreate that exact moment.

They had a file exporting with named export:

```
// greeter.js
export function getGreeting(hours) {
    // ... function body
}
```

And then tried to import it with a different name:

```
// app.js
import { greeter } from './greeter.js'
```

And the error they got was:

```
The requested module 'greeter.js' does not provide
an export named 'greeter'
```

Why did this error happen?

Because the export name was `getGreeting` but the import tried to use `greeter`. These are different names. Named exports are strict about names — they must match exactly.

The correct import was:

```
import { getGreeting } from './greeter.js'
```

This exact mistake would NOT happen with default export because with default export there is no fixed name at all. You name it whatever you want on the import side.

Adding More Named Exports to The Same File

The transcript also showed that you can have multiple named exports in one file. They added a `user` object as another export:

```
// greeter.js

export function getGreeting(hours) {
    // ... function
}

export const user = {
    name: "Rakesh",
    age: 25
}
```

Now importing:

```
import { getGreeting, user } from './greeter.js'
```

Both exported, both imported using their exact names in curly braces.

You cannot do this with default export — you can only have one default export per file. If you had tried:

```
export default function getGreeting(hours) {}
export default const user = {}  // SyntaxError - two defaults not allowed
```

This would throw an error immediately.

Mixing Both in the Same File

You can actually use BOTH in the same file. One default export and as many named exports as you want.

```
// userService.js

// Named exports
export function validateEmail(email) {
    return email.includes('@')
}
```

```
export function hashPassword(password) {
    // hashing logic
}

// Default export - the main thing this file is about
export default class UserService {
    createUser(name, email) {
        // logic
    }

    deleteUser(id) {
        // logic
    }
}
```

Importing both:

```
import UserService, { validateEmail, hashPassword } from './userService.js'
```

The default import comes **first**, before the comma. Then named imports in curly braces after the comma.

The Visual Pattern — Burn This Into Your Memory

```
NAMED EXPORT:
  Exporting:  export function myFunction() {}
  Importing:  import { myFunction } from './file.js'

            → CURLY BRACES on import
            → NAME MUST MATCH exactly
            → Can have UNLIMITED in one file

DEFAULT EXPORT:
  Exporting:  export default function myFunction() {}
  Importing:  import anything from './file.js'

            → NO CURLY BRACES on import
            → NAME CAN BE ANYTHING
            → Only ONE allowed per file
```

A Concrete Analogy to Make This Permanent

Think of a file like a shop.

Named exports are like items on a shelf with price tags and labels. Each item has a specific name label on it. If you want to buy something, you ask for it by its exact label name. You cannot ask for "the red thing" — you must say "I want the `add` function" or "I want the `subtract` function." Each item keeps its name.

Default export is like a shop that specialises in ONE main product. The shop is "The Greet Shop" and it sells greetings. When you walk in, you don't need to ask for a specific name — you just say "give me the thing you sell." You can call it whatever you want when you take it home.

Quick Decision Guide — When to Use Which

Use `export default` when:

- Your file has one clear, main purpose
- The file is essentially "about" one thing
- Example: a file that only contains one class, or one primary function

Use `export` (named) when:

- Your file provides multiple related utilities
- Multiple things are equally important
- Example: a utility file with many helper functions

Summary Table

	Named Export `export`	Default Export `export default`
Keyword	`export`	`export default`
How many per file	Unlimited	Maximum ONE
Import syntax	`import { name }`	`import anything`
Curly braces on import	YES required	NO

Name flexibility	Fixed — must match exactly	Free — choose any name
Best for	Multiple utilities	One main thing

The single most important thing to remember is this:

- Curly braces `{}` on import = named export = name must match
- No curly braces on import = default export = name is free

Once that clicks, the rest follows naturally.